

Programmng Fundamentals

Credit Hours: 3-1

Course Outline

All The Basic concepts of C++

16-Week Course Plan

Week 1

****Week 1: Introduction to C++ and Setting up the Environment****

****Lecture 1: Introduction to C++ (Theory - 50 minutes)****

Introduction to C++

Learning Objectives:

- Understand the basics of C++
- Familiarize with the C++ environment
- Learn how to compile and run C++ programs

Concepts:

C++ is a high-level, general-purpose programming language that was developed by Bjarne Stroustrup as an extension of the C language. C++ supports object-oriented programming and generic programming. C++ is a compiled language, which means that the code must be compiled before it can be executed.

To write and run C++ programs, we need a C++ compiler and a code editor or Integrated Development Environment (IDE). GCC (GNU Compiler Collection) is a popular C++ compiler. We will use Code::Blocks or Visual Studio Code as our IDE.

****Lecture 2: Setting up the Environment (Theory - 50 minutes)****

Setting up the Environment

Learning Objectives:

- Install and configure the C++ compiler and IDE
- Learn how to create a new project in the IDE
- Understand how to write and save a C++ program

Concepts:

To start programming in C++, we need to set up the environment. We will download and install the GCC compiler and a code editor or IDE. We will use Code::Blocks as our IDE.

1. Download and install Code::Blocks from the official website.
2. Download and install the GCC compiler.
3. Create a new project in Code::Blocks.
4. Write and save a simple C++ program.

****Lecture 3: Basic Syntax and Data Types (Theory - 50 minutes)****

Basic Syntax and Data Types

Learning Objectives:

- Understand the basic syntax of C++
- Learn about C++ data types
- Understand the concept of variables in C++

Concepts:

C++ syntax is similar to C syntax. It uses the same syntax for basic data types. The basic data types in C++ are:

- * `int` for integer
- * `float` for floating-point numbers
- * `char` for characters
- * `bool` for boolean values
- * `void` for no return type

Variables are used to store data in a program. In C++, we can declare variables using the `data-type variable-name;` syntax.

****Lab: Setting up the Environment and Writing a C++ Program (Lab - 50 minutes)****

Lab Objectives:

- Set up the C++ environment
- Write and run a C++ program
- Understand how to use variables in C++

Lab Instructions:

1. Set up the C++ environment using the instructions provided in the lecture.
2. Create a new project in Code::Blocks.
3. Write a simple C++ program that uses variables and data types.
4. Run the program and observe the output.

Week 2

****Week 2: Data Types and Variables****

Lecture 1: Introduction to C++ Data Types (1.5 hours)

Learning Objectives

- * Understand the basic data types in C++
- * Learn how to declare and use variables in C++

Concepts

C++ supports various data types, including:

- * Integers: ``int``, ``char`` (single character)
- * Floating-point numbers: ``float``, ``double``, ``long double``
- * Boolean values: ``bool``
- * Character strings: ``string`` (introduced in C++11)

Variables are declared using the ``data_type variable_name;`` syntax. For example:

```
```c
int x = 10; // declare and initialize an integer variable
double pi = 3.14; // declare and initialize a double variable
bool isAdmin = true; // declare and initialize a boolean variable
```
```

Lecture 2: Variables and Assignments (1.5 hours)

Learning Objectives

- * Learn how to assign values to variables
- * Understand the concept of variable scope and lifetime

Concepts

Variables can be assigned values using the assignment operator (``=``):

```
```c
x = 20; // assign a new value to the variable x
pi = 3.14159; // assign a new value to the variable pi
```
```

Variable scope refers to the region of the code where a variable is accessible. A variable's lifetime is the period during which it exists in memory.

Lecture 3: Operators and Expressions (1.5 hours)

Learning Objectives

- * Learn how to use operators in C++
- * Understand how to evaluate expressions in C++

Concepts

C++ supports various operators, including:

- * Arithmetic operators: ``+``, ``-``, ``*``, ``/``, etc.

* Comparison operators: `==`, `!=`, `>`, `<`, etc.

* Logical operators: `&&`, `||`, `!`, etc.

Expressions are combined using operators. For example:

```
```c
```

```
x = 5 + 3; // evaluate an expression using the addition operator
```

```
result = x > 10; // evaluate an expression using the comparison operator
```

```
```
```

Lab 2: Data Types and Variables

Exercise 1

Write a C++ program to declare and initialize variables of different data types.

Exercise 2

Write a C++ program to assign values to variables using the assignment operator.

Exercise 3

Write a C++ program to evaluate expressions using various operators.

Week 3

****Week 3: Control Structures****

Lecture 1: Conditional Statements (1.5 hours)

Conditional Statements

Learning Objectives

* Understand the concept of conditional statements

* Learn to use if-else statements in C++

* Learn to use switch statements in C++

Concepts

Conditional statements are used to execute different blocks of code based on certain conditions. In C++, you can use if-else statements to achieve this. The if-else statement consists of an if clause and an optional else clause. The if clause is used to specify the condition and the else clause is used to specify the code to be executed if the condition is not met.

```
```c
```

```
if (condition) {
```

```
// code to be executed if condition is true
```

```
} else {
```

```
// code to be executed if condition is false
}
...
```

You can also use switch statements to execute different blocks of code based on the value of a variable. The switch statement consists of a switch keyword followed by a variable and a series of case labels.

```
```c  
switch (variable) {  
    case label1:  
        // code to be executed if variable equals label1  
        break;  
    case label2:  
        // code to be executed if variable equals label2  
        break;  
    default:  
        // code to be executed if variable does not match any case  
        break;  
}  
...
```

Lecture 2: Loops (1.5 hours)

Loops

Learning Objectives

- * Understand the concept of loops
- * Learn to use for loops in C++
- * Learn to use while loops in C++
- * Learn to use do-while loops in C++

Concepts

Loops are used to execute a block of code repeatedly for a specified number of times. In C++, you can use for loops, while loops, and do-while loops to achieve this. The for loop is used to execute a block of code for a specified number of times. The while loop is used to execute a block of code while a certain condition is true. The do-while loop is used to execute a block of code at least once and then repeatedly while a certain condition is true.

```
```c
```

```

// for loop
for (int i = 0; i < 5; i++) {
 // code to be executed for each iteration
}

// while loop
int i = 0;
while (i < 5) {
 // code to be executed while condition is true
 i++;
}

// do-while loop
int i = 0;
do {
 // code to be executed at least once
 i++;
} while (i < 5);
...

```

### **Lecture 3: Loop Control Statements (1 hour)**

#### **Loop Control Statements**

##### **Learning Objectives**

- \* Learn to use break statements in C++
- \* Learn to use continue statements in C++
- \* Learn to use exit statements in C++

##### **Concepts**

Loop control statements are used to control the flow of a loop. In C++, you can use break statements to exit a loop prematurely, continue statements to skip the current iteration of a loop, and exit statements to exit a program prematurely.

```
```c
```

```

// break statement
for (int i = 0; i < 5; i++) {
    if (i == 3) {

```

```

break;
}
// code to be executed for each iteration
}
// continue statement
for (int i = 0; i < 5; i++) {
    if (i == 3) {
        continue;
    }
    // code to be executed for each iteration
}
// exit statement
exit(0);
...

```

Lab: Control Structures (1 hour)

- * Write a program that uses if-else statements to determine whether a number is even or odd.
- * Write a program that uses switch statements to determine the day of the week based on a given day number.
- * Write a program that uses for loops to print the numbers from 1 to 10.
- * Write a program that uses while loops to print the numbers from 1 to 10.
- * Write a program that uses do-while loops to print the numbers from 1 to 10.

Week 4

****Week 4: Functions and Control Structures****

Lecture 1: Functions (1.5 hours)

Introduction to Functions

Learning Objectives

- * Define a function in C++
- * Explain the benefits of using functions
- * Identify the different types of functions

Concepts

A function is a block of code that can be called multiple times from different parts of the program. It takes arguments, performs a specific task, and returns a value. Functions help in reducing code duplication, improving code organization, and making the program more modular. In C++, functions can be defined using the `return` type and the function name.

Example

```
```cpp
int add(int a, int b) {
 return a + b;
}
```
```

Lecture Notes

- * Functions can be declared and defined separately.
- * Functions can take arguments and return values.
- * Functions can be overloaded, allowing multiple functions with the same name but different parameters.

Lecture 2: Function Arguments and Return Types (1.5 hours)

Function Arguments and Return Types

Learning Objectives

- * Explain the different types of function arguments
- * Identify the different types of function return types
- * Use function arguments and return types in C++

Concepts

Function arguments are the values passed to a function when it is called. They can be passed by value or by reference. Function return types specify the type of value that a function returns. In C++, functions can return different types of values, including integers, characters, and arrays.

Example

```
```cpp
int max(int a, int b) {
 if (a > b) {
 return a;
 } else {
 return b;
 }
}
```

```
}
...

```

### Lecture Notes

- \* Functions can take multiple arguments.
- \* Functions can return multiple values using pointers or arrays.

### Lecture 3: Control Structures - Conditional Statements (1.5 hours)

#### Conditional Statements

##### Learning Objectives

- \* Explain the different types of conditional statements
- \* Identify the syntax of if-else statements
- \* Use conditional statements in C++

##### Concepts

Conditional statements are used to control the flow of a program based on certain conditions. In C++, the most common conditional statements are if-else statements. If-else statements allow the program to execute different blocks of code based on a condition.

##### Example

```
```cpp  
  
int x = 5;  
  
if (x > 10) {  
    cout << "x is greater than 10";  
}  
else {  
    cout << "x is less than or equal to 10";  
}  
}  
...  

```

Lecture Notes

- * Conditional statements can be used to make decisions in a program.
- * Conditional statements can be nested to make more complex decisions.

Lab: Functions and Control Structures

Lab Objectives

- * Write a program that uses functions and conditional statements.
- * Use functions to perform calculations and conditional statements to make decisions.

Lab Instructions

Write a program that asks the user for their age and then uses a function to calculate their age group. The program should use conditional statements to determine the age group based on the user's age.

Note: The lab instructions will be provided separately.

Week 5

****Week 5: Control Flow in C++ (3 lectures, 1 lab)****

****Lecture 1: Conditional Statements (1 hour)****

Lecture Title: Conditional Statements

Learning Objectives:

- Understand the concept of conditional statements
- Learn to use if-else statements in C++
- Learn to use switch statements in C++

Concepts:

Conditional statements are used to execute different blocks of code based on a condition. In C++, we can use the if-else statement to execute different blocks of code based on a condition. The basic syntax of the if-else statement is:

```
```c
if (condition) {
 // code to execute if condition is true
} else {
 // code to execute if condition is false
}
```
```

The switch statement is used to execute different blocks of code based on the value of a variable. The basic syntax of the switch statement is:

```
```c
switch (expression) {
 case value1:
 // code to execute if expression is equal to value1
 break;
 case value2:
```

```

// code to execute if expression is equal to value2

break;

default:

// code to execute if expression does not match any value

break;

}

...

```

**\*\*Lecture 2: Loops in C++ (1 hour)\*\***

### **Lecture Title: Loops in C++**

#### **Learning Objectives:**

- Understand the concept of loops in C++
- Learn to use for loops in C++
- Learn to use while loops in C++

#### **Concepts:**

Loops are used to execute a block of code repeatedly for a specified number of times. In C++, we can use two types of loops: for loops and while loops.

A for loop is used to execute a block of code for a specified number of times. The basic syntax of a for loop is:

```

```c
for (initialization; condition; increment) {

// code to execute

}

...

```

A while loop is used to execute a block of code as long as a specified condition is true. The basic syntax of a while loop is:

```

```c
while (condition) {

// code to execute

}

...

```

**\*\*Lecture 3: Loop Control Statements (1 hour)\*\***

### **Lecture Title: Loop Control Statements**

### Learning Objectives:

- Understand the concept of loop control statements
- Learn to use break and continue statements in C++
- Learn to use labels in C++

### Concepts:

Loop control statements are used to control the flow of a loop. In C++, we can use break and continue statements to control the flow of a loop.

The break statement is used to terminate a loop prematurely.

```
```c
while (condition) {
    if (break condition) {
        break;
    }
}
```
```

The continue statement is used to skip the remaining code in a loop and move to the next iteration.

```
```c
while (condition) {
    if (continue condition) {
        continue;
    }
}
```
```

Labels are used to specify a label for a loop. We can use the goto statement to jump to a labeled loop.

```
```c
label1:
while (condition) {
    // code to execute
    goto label1;
}
```
```

**\*\*Lab:\*\***

**\*\*Lab Title: Control Flow in C++\*\***

### **Lab Objectives:**

- Write a C++ program that uses if-else statements to execute different blocks of code based on a condition.
- Write a C++ program that uses switch statements to execute different blocks of code based on the value of a variable.
- Write a C++ program that uses for and while loops to execute a block of code repeatedly for a specified number of times.
- Write a C++ program that uses break and continue statements to control the flow of a loop.

### **Lab Instructions:**

1. Write a C++ program that asks the user for their age and checks if they are eligible to vote. If the user is eligible to vote, print a message saying "You are eligible to vote." If the user is not eligible to vote, print a message saying "You are not eligible to vote."
2. Write a C++ program that uses a switch statement to execute different blocks of code based on the value of a variable. The variable should be a character and the program should print the corresponding alphabet.
3. Write a C++ program that uses a for loop to print the numbers from 1 to 10.
4. Write a C++ program that uses a while loop to print the numbers from 1 to 10.
5. Write a C++ program that uses break and continue statements to control the flow of a loop. The program should print the numbers from 1 to 10, but skip the numbers that are divisible by 3.

## **Week 6**

**\*\*Week 6: File Input/Output, Pointers, and Arrays in C++\*\***

### **Lecture 1: Introduction to File Input/Output (30 minutes)**

#### **Lecture Title: File Input/Output Basics**

#### **Learning Objectives:**

- Understand the concept of file input/output in C++.
- Learn to read and write text files.
- Understand the difference between text and binary files.

#### **Concepts:**

In C++, file input/output is a crucial aspect of programming. We can interact with files using streams, which are objects that allow us to read and write data to files. There are two types of files: text files and binary files. Text files contain human-readable data, while binary files contain raw binary data. To read and write files, we use the `ifstream` and `ofstream` classes, respectively. The `>>` operator is used to extract data from the input stream, while the `<<` operator is used to insert

data into the output stream. We will also learn about file modes, such as `ios::in` for reading and `ios::out` for writing.

## **Lecture 2: File Input/Output Operations (30 minutes)**

### **Lecture Title: File Input/Output Operations**

#### **Learning Objectives:**

- Learn how to open files for input and output.
- Understand how to read and write data to files.
- Learn about file modes and error handling.

#### **Concepts:**

To perform input/output operations on a file, we use the `open()` function, which takes two arguments: the filename and the file mode. The `ios::in` mode is used for input, while the `ios::out` mode is used for output. We can also use the `ios::in | ios::out` mode to open a file for both reading and writing. The `is_open()` function checks if the file is open, and the `close()` function closes the file. Error handling is crucial when working with files, and we use the `fail()` function to check for errors.

## **Lecture 3: Pointers and Arrays (30 minutes)**

### **Lecture Title: Pointers and Arrays**

#### **Learning Objectives:**

- Understand the concept of pointers in C++.
- Learn how to declare and use pointers.
- Understand the difference between arrays and pointers.

#### **Concepts:**

In C++, a pointer is a variable that stores the memory address of another variable. We declare a pointer using the asterisk symbol (`*`). Pointers are useful for dynamic memory allocation and passing variables to functions. Arrays are collections of variables of the same data type stored in contiguous memory locations. We can access array elements using the index operator (`[]`). However, arrays and pointers are not the same, and we can use pointers to simulate arrays using dynamic memory allocation.

**\*\*Lab:\*\***

#### **Lab Assignment:**

- Create a program that reads a text file and writes its contents to a new file.
- Write a program that uses pointers to simulate an array of integers.
- Write a program that uses file input/output to read and write binary data.

Note: The lab assignments are designed to reinforce the concepts learned in the lectures and provide hands-on experience with file input/output and pointers in C++.

## Week 7

**\*\*Week 7: Classes and Objects\*\***

### **Lecture 1: Introduction to Classes and Objects (Theory)**

##### Learning Objectives:

- Understand the concept of classes and objects in C++
- Learn how to define and use classes and objects

##### Concepts:

In C++, a class is a blueprint or a template that defines the properties and behavior of an object. An object is an instance of a class and has its own set of attributes (data) and methods (functions). Classes are used to encapsulate data and behavior, making it easier to organize and reuse code.

```
```cpp
class Car {
private:
    string color;
    int speed;
public:
    void setSpeed(int s) { speed = s; }
    int getSpeed() { return speed; }
};
```
```

### **Lecture 2: Member Functions and Access Modifiers (Theory)**

##### Learning Objectives:

- Understand the concept of member functions and access modifiers in C++
- Learn how to define and use member functions and access modifiers

##### Concepts:

Member functions are functions that are defined inside a class and operate on the data members of the class. Access modifiers (public, private, protected) control the access to data members and member functions.

```
```cpp
class BankAccount {
private:
    double balance;
```



```

public:

void deposit(double amount) { balance += amount; }

double getBalance() { return balance; }

};
...

```

Lecture 3: Constructors and Destructors (Theory)

Learning Objectives:

- Understand the concept of constructors and destructors in C++
- Learn how to define and use constructors and destructors

Concepts:

Constructors are special member functions that are called when an object is created. Destructors are called when an object is destroyed. Constructors and destructors are used to initialize and clean up objects.

```

```cpp
class Person {
public:
 Person(string name) { this->name = name; }
 ~Person() { cout << "Person object destroyed"; }
private:
 string name;
};
...

```

### **Lab 7: Classes and Objects Lab**

##### Exercise:

Create a class `Vehicle` with attributes `color`, `speed`, and `capacity`. Implement member functions to set and get these attributes. Create an object `myVehicle` and demonstrate how to use the member functions.

## **Week 8**

**\*\*Week 8: Exception Handling and File Input/Output\*\***

### **Lecture 1: Exception Handling (Theory - 50 minutes)**

#### **Exception Handling Basics**

##### **Learning Objectives**

- \* Understand the concept of exceptions in C++
- \* Learn how to handle exceptions using try-catch blocks

### Concepts

In C++, exceptions are events that occur during the execution of a program, such as division by zero or out-of-range values. The `try` block is used to enclose the code that might throw an exception, while the `catch` block is used to handle the exception. The `throw` keyword is used to throw an exception.

```
```cpp
try {
    // code that might throw an exception
} catch (exception& e) {
    // handle the exception
}
...

```

Lecture 2: Exception Handling (Theory - 50 minutes)

Exception Handling Best Practices

Learning Objectives

- * Learn how to handle specific exceptions using catch blocks
- * Understand how to rethrow exceptions

Concepts

To handle specific exceptions, we can use different `catch` blocks to catch exceptions of different types. We can also rethrow exceptions using the `throw` keyword.

```
```cpp
try {
 // code that might throw an exception
} catch (std::runtime_error& e) {
 // handle runtime error
} catch (std::logic_error& e) {
 // handle logic error
}
...

```

### Lecture 3: Exception Handling (Theory - 50 minutes)

## Exception Handling in Real-World Scenarios

### Learning Objectives

- \* Learn how to use exceptions in real-world scenarios, such as file input/output
- \* Understand how to handle exceptions in a user-friendly manner

### Concepts

Exceptions are not just for division by zero errors. We can also use exceptions to handle file input/output errors, such as file not found or permission denied errors.

```
```cpp
try {
    std::ifstream file("example.txt");

    // read from file

} catch (std::ifstream::failure& e) {
    // handle file input/output error
}
```
```

### Lab: Exception Handling and File Input/Output

**Lab Description:** Implement exception handling in a C++ program that reads from a file. Handle specific exceptions, such as file not found or permission denied errors.

**Deliverables:**

- \* A C++ program that reads from a file and handles exceptions
- \* A written report on the implementation of exception handling in the program

## Week 9

**Week 9: Advanced Data Structures**

### Lecture 1: Linked Lists (Theory)

#### Linked Lists

### Learning Objectives

- \* Understand the concept of linked lists
- \* Learn how to implement a basic linked list
- \* Understand the advantages and disadvantages of linked lists

### Concepts

A linked list is a dynamic data structure in which each element is a separate object, known as a node. Each node contains two items: the data and a reference (or link) to the next node in the sequence. This structure allows for efficient insertion or removal of elements from any position in the sequence.

## **Lecture 2: Stacks and Queues (Theory)**

### **Stacks and Queues**

#### **Learning Objectives**

- \* Understand the concept of stacks and queues
- \* Learn how to implement a stack and a queue using arrays or linked lists
- \* Understand the applications of stacks and queues

#### **Concepts**

A stack is a Last-In-First-Out (LIFO) data structure, where the last element added is the first one to be removed. A queue is a First-In-First-Out (FIFO) data structure, where the first element added is the first one to be removed. Both stacks and queues can be implemented using arrays or linked lists.

## **Lecture 3: Trees and Graphs (Theory)**

### **Trees and Graphs**

#### **Learning Objectives**

- \* Understand the concept of trees and graphs
- \* Learn how to represent trees and graphs using adjacency matrices or adjacency lists
- \* Understand the basic operations on trees and graphs

#### **Concepts**

A tree is a hierarchical data structure consisting of nodes with a parent-child relationship. A graph is a non-hierarchical data structure consisting of nodes with edges connecting them. Trees and graphs are used to represent complex relationships between data.

## **Lab 1: Implementing Linked Lists and Stacks/Queues**

### **Lab Exercise**

#### **Learning Objectives**

- \* Implement a linked list and a stack/queue using C++
- \* Understand how to test and debug your implementation

#### **Concepts**

In this lab, you will implement a linked list and a stack/queue using C++. You will also learn how to test and debug your implementation using a debugger or print statements.

## Week 10

**\*\*Week 10: Exception Handling and File Input/Output\*\***

### **Lecture 1: Exception Handling (≈150 words)**

#### Handling Exceptions in C++

##### **Learning Objectives**

- Understand the concept of exceptions in C++
- Learn how to throw and catch exceptions
- Use try-catch blocks to handle exceptions

##### **Concepts**

In C++, exceptions are used to handle runtime errors. An exception is an event that occurs during the execution of a program, such as a division by zero or an out-of-range array index. The `try` block contains code that might throw an exception, while the `catch` block handles the exception. To throw an exception, use the `throw` keyword. To catch an exception, use a `catch` block with a parameter to hold the exception object.

### **Lecture 2: Exception Handling (≈150 words)**

#### Advanced Exception Handling

##### **Learning Objectives**

- Learn how to handle multiple exceptions in a single `catch` block
- Understand the concept of exception specifications
- Use `std::exception` and `std::runtime_error` classes to handle exceptions

##### **Concepts**

Multiple exceptions can be caught in a single `catch` block by using a `catch` block with a parameter of type `std::exception`. Exception specifications are used to specify the types of exceptions that a function can throw. The `std::exception` class is a base class for all exceptions, while `std::runtime_error` is a specific exception class for runtime errors.

### **Lecture 3: Exception Handling (≈150 words)**

#### Best Practices for Exception Handling

##### **Learning Objectives**

- Learn how to avoid exceptions in code
- Understand the importance of exception handling
- Use `std::cerr` to print error messages

##### **Concepts**

Exceptions should be avoided in code by checking for potential errors before executing potentially hazardous code. Exception handling is essential in C++ to handle runtime errors. The `std::cerr` stream is used to print error messages.

### **Lab: Exception Handling and File Input/Output (≈150 words)**

#### **Lab Assignment**

##### **Learning Objectives**

- Implement exception handling in a C++ program
- Use file input/output to read and write data
- Handle file-related exceptions

##### **Concepts**

In this lab, you will implement exception handling in a C++ program that reads and writes data to a file. Use file input/output to read and write data, and handle file-related exceptions using try-catch blocks.

#### **Week 11**

**\*\*Week 11: File Input/Output and Exception Handling\*\***

### **Lecture 1: Introduction to File Input/Output (50 minutes)**

#### **Lecture Title: File Input/Output Basics**

##### **Learning Objectives:**

- Understand the concept of file input/output in C++
- Learn to read and write data to files
- Understand the differences between text and binary files

##### **Concepts:**

In C++, file input/output is a fundamental concept that allows you to read and write data to files. There are two types of files: text files and binary files. Text files contain plain text data, while binary files contain raw binary data.

You can use the `fstream` class to perform file I/O operations. To open a file, you use the `open` function, and to read or write data, you use the `read` and `write` functions, respectively.

```
```cpp
#include

using namespace std;

int main() {

    ifstream inputFile; // input file stream object

    ofstream outputFile; // output file stream object
```

```

inputFile.open("input.txt"); // open input file

outputFile.open("output.txt"); // open output file

// read data from input file and write to output file

char data;

while (inputFile.get(data)) {

    outputFile.put(data);

}

inputFile.close();

outputFile.close();

return 0;

}

...

```

Lecture 2: Exception Handling (50 minutes)

Lecture Title: Exception Handling

Learning Objectives:

- Understand the concept of exceptions in C++
- Learn to use try-catch blocks to handle exceptions
- Understand the differences between try, catch, and throw

Concepts:

Exception handling is a mechanism in C++ that allows you to handle runtime errors and exceptions. An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.

To handle exceptions, you use try-catch blocks. The `try` block contains the code that may throw an exception, and the `catch` block contains the code that handles the exception.

```

```cpp

#include

int main() {

 int x = 5;

 int y = 0;

 try {

 // code that may throw an exception

 int result = x / y;
 }
}

```

```

 } catch (const std::runtime_error& e) {
 // handle the exception
 cout << "Error: " << e.what() << endl;
 }
 return 0;
}
...

```

### Lecture 3: Advanced Exception Handling (50 minutes)

#### Lecture Title: Advanced Exception Handling Techniques

##### Learning Objectives:

- Learn to use multiple catch blocks to handle different types of exceptions
- Understand how to rethrow exceptions
- Learn about the `throw` statement

##### Concepts:

You can use multiple catch blocks to handle different types of exceptions. The `throw` statement allows you to throw an exception explicitly.

```

```cpp
#include

int main() {
    int x = 5;
    int y = 0;
    try {
        // code that may throw an exception
        int result = x / y;
    } catch (const std::runtime_error& e) {
        // handle runtime error
        cout << "Error: " << e.what() << endl;
    } catch (const std::invalid_argument& e) {
        // handle invalid argument error
        cout << "Error: " << e.what() << endl;
    }
}

```



```
return 0;
```

```
}
```

```
...
```

****Lab:****

Lab Exercise: File Input/Output and Exception Handling

1. Write a C++ program that reads a text file and writes its contents to another text file.
2. Implement exception handling in the program to handle file I/O errors.
3. Use a try-catch block to handle the exception and print an error message.

Note: The lab exercise requires you to write code and implement the concepts learned in the lectures.

Week 12

****Week 12****

Lecture 1: Exception Handling (Theory)

Exception Handling in C++

Learning Objectives

- * Understand the concept of exceptions in C++
- * Learn how to handle exceptions using try-catch blocks
- * Understand the difference between synchronous and asynchronous exceptions

Concepts

In C++, exceptions are used to handle runtime errors. They are triggered when an error occurs during the execution of a program. The `try` block is used to enclose code that may potentially throw an exception. The `catch` block is used to handle the exception. There are two types of exceptions: synchronous and asynchronous. Synchronous exceptions occur at the point where the exception is thrown, while asynchronous exceptions occur at a later point.

Lecture 2: Exception Handling (Theory)

Exception Class Hierarchy

Learning Objectives

- * Understand the exception class hierarchy in C++
- * Learn how to use the `std::exception` class
- * Understand how to create custom exception classes

Concepts

The `std::exception` class is the base class for all exceptions in C++. It provides virtual functions for getting the exception's message and `what()`. Custom exception classes can be derived from `std::exception` to create specific exception types. The `what()` function returns a string describing the exception.

Lecture 3: Exception Handling (Theory)

Exception Handling Best Practices

Learning Objectives

- * Learn best practices for handling exceptions in C++
- * Understand how to avoid throwing exceptions in performance-critical code
- * Learn how to handle exceptions in a multi-threaded environment

Concepts

Best practices for handling exceptions include avoiding throwing exceptions in performance-critical code, using `std::exception` as the base class for custom exceptions, and handling exceptions in a multi-threaded environment. Exceptions should be thrown only when necessary, and they should be handled as close to the point of occurrence as possible.

Lab: Exception Handling

Lab Exercise

Implement a program that demonstrates exception handling using try-catch blocks. The program should throw a custom exception when a user enters an invalid input.

Week 13

****Week 13: Exception Handling and File Input/Output****

Lecture 1: Introduction to Exception Handling (≈75 words)

Exception Handling Basics

Learning Objectives:

- * Understand the need for exception handling in C++
- * Identify and explain different types of exceptions
- * Write basic try-catch blocks to handle exceptions

Concepts:

In C++, exceptions occur when an error or unexpected event occurs during the execution of a program. The try-catch block is used to handle exceptions. The try block contains code that may throw an exception, while the catch block contains code that will be executed if an exception occurs. The `throw` keyword is used to throw an exception.

Lecture 2: Exception Handling in C++ (≈75 words)

Exception Handling in C++

Learning Objectives:

- * Understand how to use the `try` and `catch` blocks to handle exceptions
- * Learn about the `throw` keyword and its usage
- * Understand the differences between synchronous and asynchronous exceptions

Concepts:

In C++, exceptions can be synchronous or asynchronous. Synchronous exceptions occur at the point where the error occurs, while asynchronous exceptions occur at a later point in the program. The `throw` keyword is used to throw an exception. The `try` and `catch` blocks are used to handle exceptions.

Lecture 3: File Input/Output in C++ (~75 words)

File Input/Output in C++

Learning Objectives:

- * Understand the basics of file input/output in C++
- * Learn how to open and close files
- * Understand how to read and write data to files

Concepts:

In C++, files are used to store data permanently. The `fstream` class is used to perform file input/output operations. The `open` function is used to open a file, while the `close` function is used to close a file. The `read` and `write` functions are used to read and write data to files.

Lab 1: Exception Handling and File Input/Output (~150 words)

Lab: Exception Handling and File Input/Output

Learning Objectives:

- * Practice exception handling using try-catch blocks
- * Practice file input/output using the `fstream` class
- * Apply exception handling and file input/output concepts to a real-world scenario

Concepts:

In this lab, you will practice exception handling using try-catch blocks and file input/output using the `fstream` class. You will apply these concepts to a real-world scenario, such as reading and writing data to a file.

Week 14

****Week 14: Exception Handling and File Input/Output****

Lecture 1: Introduction to Exception Handling

Learning Objectives:

- * Understand the concept of exceptions and exception handling in C++
- * Learn how to use try-catch blocks to handle exceptions

Concepts:

In C++, exceptions are events that occur during the execution of a program that disrupt the normal flow of instructions. Exception handling is a mechanism to handle these exceptions and prevent the program from crashing. The try-catch block is used to handle exceptions. The try block contains the code that may raise an exception, while the catch block contains the code that handles the exception.

```
```cpp
try {
 // code that may raise an exception
} catch (exception_type e) {
 // handle the exception
}
...

```

## Lecture 2: Exception Types and Handling

#### #### Learning Objectives:

- \* Understand the different types of exceptions in C++
- \* Learn how to handle common exceptions such as divide by zero, out of range, and invalid argument

#### #### Concepts:

There are two types of exceptions in C++: standard exceptions and user-defined exceptions. Standard exceptions include divide by zero, out of range, and invalid argument, among others. These exceptions can be handled using the try-catch block.

```
```cpp
try {
    int x = 5;
    int y = 0;
    int result = x / y;
} catch (std::runtime_error& e) {
    std::cerr << "Exception caught: " << e.what() << std::endl;
}

```

...

Lecture 3: File Input/Output

Learning Objectives:

- * Understand the concept of file input/output in C++
- * Learn how to read and write files using the istream and ostream classes

Concepts:

File input/output in C++ is used to read and write data to files. The istream class is used to read data from a file, while the ostream class is used to write data to a file. The file is opened using the open() function and data is read or written using the read() and write() functions.

```
```cpp
```

```
#include

int main() {

 std::ifstream inputFile("input.txt");

 std::ofstream outputFile("output.txt");

 if (inputFile.is_open() && outputFile.is_open()) {

 char c;

 while (inputFile.get(c)) {

 outputFile.put(c);

 }

 }

 inputFile.close();

 outputFile.close();

 return 0;

}
```

...

### Lab: Exception Handling and File Input/Output

#### #### Objective:

- \* Write a C++ program that reads a file, catches exceptions, and writes the contents to another file.

#### #### Task:

Write a C++ program that reads a file named "input.txt", catches any exceptions that occur during the reading process, and writes the contents to a file named "output.txt".

## **\*\*Week 15: Exception Handling and File Input/Output\*\***

### **Lecture 1: Introduction to Exception Handling**

#### **Learning Objectives:**

- Understand the need for exception handling in C++
- Learn to use try-catch blocks to handle exceptions

#### **Concepts:**

In C++, exceptions are events that occur during the execution of a program that disrupt the normal flow of the program. Exception handling is a mechanism to handle such events. The try-catch block is used to handle exceptions.

```
```cpp
try {
    // code that may throw an exception
} catch (exception_type e) {
    // code to handle the exception
}
```
```

### **Lecture 2: Exception Handling in C++**

#### **Learning Objectives:**

- Learn to use specific exception types in C++
- Understand the concept of exception propagation

#### **Concepts:**

C++ provides a variety of exception classes that can be used to handle different types of exceptions. Some of these exception classes include: `std::runtime_error`, `std::invalid_argument`, `std::out_of_range`.

```
```cpp
try {
    throw std::runtime_error("Invalid input");
} catch (const std::runtime_error& e) {
    std::cerr << "Runtime error: " << e.what() << std::endl;
}
```
```

### **Lecture 3: File Input/Output in C++**

### Learning Objectives:

- Learn to read and write files in C++
- Understand the use of file streams

### Concepts:

C++ provides classes `fstream` and `iostream` for file input/output operations. The `ifstream` class is used to read from a file, while the `ofstream` class is used to write to a file.

```
```cpp
#include
#include

int main() {
    std::ifstream inputFile("input.txt");
    if (inputFile.is_open()) {
        std::string line;
        while (std::getline(inputFile, line)) {
            std::cout << line << std::endl;
        }
        inputFile.close();
    }
    return 0;
}
```
```

### Lab: Exception Handling and File Input/Output

#### Learning Objectives:

- Practice exception handling and file I/O operations
- Implement a program that handles exceptions and reads/writes a file

#### Concepts:

Implement a program that reads a file, checks for exceptions, and writes the contents of the file to the console. The program should handle the following exceptions: `std::runtime_error`, `std::invalid_argument`, `std::out_of_range`.

## Week 16

**\*\*Week 16: Exception Handling and Advanced Topics\*\***

### Lecture 1: Exception Handling (≈150 words)

## Exception Handling in C++

### Learning Objectives

- Understand the concept of exceptions in C++
- Learn how to handle exceptions using try-catch blocks
- Understand the difference between exceptions and errors

### Concepts

In C++, exceptions are a way to handle runtime errors that occur during the execution of a program. The `try` block is used to enclose code that might throw an exception, while the `catch` block is used to handle the exception. The `throw` keyword is used to throw an exception. Exceptions are used to provide a way to handle errors in a more controlled manner than using error codes.

```
```cpp
try {
    // Code that might throw an exception
} catch (exception& e) {
    // Handle the exception
}
```
```

## Lecture 2: Advanced Topics - Smart Pointers (≈150 words)

### Smart Pointers in C++

#### Learning Objectives

- Understand the concept of smart pointers in C++
- Learn how to use `unique_ptr` and `shared_ptr` to manage memory
- Understand the benefits of using smart pointers

#### Concepts

Smart pointers are a type of pointer that automatically manage the memory they point to. They are used to prevent memory leaks and make code more exception-safe. `unique_ptr` is a unique pointer that owns and manages another object through a pointer. `shared_ptr` is a shared pointer that manages shared ownership of an object.

```
```cpp
std::unique_ptr ptr(new int(5));
std::shared_ptr ptr(new int(5));
```
```



### Lecture 3: Advanced Topics - Lambda Functions (≈150 words)

#### Lambda Functions in C++

##### Learning Objectives

- Understand the concept of lambda functions in C++
- Learn how to use lambda functions to create small, anonymous functions
- Understand the benefits of using lambda functions

##### Concepts

Lambda functions are small, anonymous functions that can be defined inline within a larger expression. They are used to create small, one-time use functions. Lambda functions are defined using the `[]` operator and can capture variables from the surrounding scope.

```
```cpp
auto lambda = [] (int x) { return x * x; };
```
```

### Lab 1: Exception Handling and Smart Pointers (≈150 words)

#### Lab: Exception Handling and Smart Pointers

##### Learning Objectives

- Practice using exception handling and smart pointers in C++
- Understand how to use `try-catch` blocks and `unique_ptr` and `shared_ptr`
- Apply the concepts learned in the lectures to a real-world scenario

##### Concepts

In this lab, you will practice using exception handling and smart pointers to manage memory and handle runtime errors. You will write a program that uses `try-catch` blocks to handle exceptions and `unique_ptr` and `shared_ptr` to manage memory.